An EDSL-Defined Loop for School Choice

The Machine's State, Iteration, and Execution

Technical note for the LLM preference pilot

October 11, 2026

Abstract

This note documents the revised school-choice Machine, in which deferred acceptance's proposer selection, proposal construction, state update, and stop condition are all represented as EDSL data expressions. A generic EDSL Runtime initializes the Machine and applies its standard close transition; the close effect evaluates a bounded `iterate` expression until the matching algorithm's stop condition holds. There is no experiment-specific Python loop selecting proposals. The Machine remains JSON-serializable and declares no Python algorithm callbacks. This design is an important step toward submitting an experiment to a generic remote executor: market-specific procedure is now in the serialized definition, while the EDSL interpreter remains the shared execution engine.

1 The change in one sentence

The earlier version exposed one command per proposal and left Python to select each next proposer. The current version puts the complete deferred-acceptance loop inside the Machine's `close_effects`: an EDSL `iterate` repeatedly chooses the next proposer, computes the proposal transition, and stops when no proposer remains eligible. The experiment runner now just constructs a Machine, initializes the generic Runtime, and invokes the standard close operation.

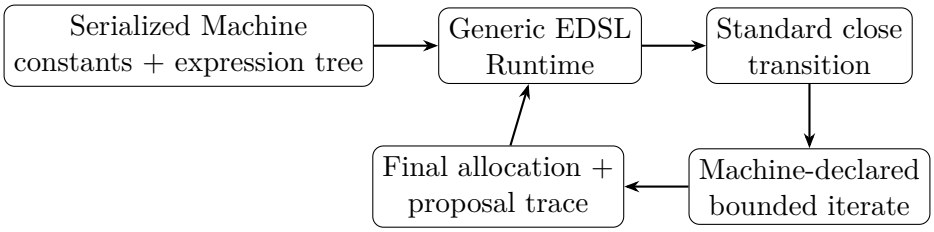


Figure 1: The generic Runtime executes the loop encoded in the Machine. Python no longer chooses each proposer.

2 What is serialized

`build_machine` runs at authoring time. It selects one proposer side, validates the supplied student rankings, computes rank lookup maps, embeds preferences, capacities, and school priorities as constants, and constructs the expression tree for that side's loop. `Machine.to_json()` serializes those constants and EDSL expressions; `algorithms=()` means no named Python callback is required to execute the mechanism.

Listing 1: Machine definition: serialized constants, state type, close effect, and completion condition

```
1
```

```python
def build_machine(
    seed: int = 42017,
    proposer_side: str = "student",
    student_preferences_override=None,
) -> Machine:
    """Build a Machine whose close transition runs DA entirely in the DSL."""
    if proposer_side not in {"student", "school"}:
        raise ValueError("proposer_side must be 'student' or 'school'")

    preferences, student_ranks, priorities, priority_ranks = make_preferences(
        seed, student_preferences_override
    )
    side_name = f"{proposer_side}-proposing"
    capacities = {school: CAPACITY for school in SCHOOLS}
    initial = _initial_allocation()
    machine = Machine(
        name=f"SchoolChoiceDeferredAcceptance_{proposer_side.title()}Proposing",
        constants={
            "seed": seed,
            "proposer_side": side_name,
            "students": list(STUDENTS),
            "schools": list(SCHOOLS),
            "capacities": capacities,
            "initial_allocation": initial,
            "student_preferences": preferences,
            "student_ranks": student_ranks,
            "school_priorities": priorities,
            "school_priority_ranks": priority_ranks,
            "total_capacity": sum(capacities.values()),
        },
        fields={"allocation": state_field(_allocation_type(), initial)},
        commands={},
        view={
            "proposer_side": constant("proposer_side"),
            "assignments": field("allocation").get("assignments"),
            "school_holds": field("allocation").get("school_holds"),
            "proposal_log": field("allocation").get("proposal_log"),
            "finished": field("allocation").get("finished"),
            "current_identity": current("name"),
        },
        close_effects=(
            assign(
                "allocation",
                _student_proposing_loop()
                if proposer_side == "student"
                else _school_proposing_loop(),
            ),
        ),
        complete_when=field("allocation").get("finished"),
        algorithms=(),
    )
    machine.validate()
```

The serialized definition is portable between EDSL runtimes that implement the declared Machine language version and expression capabilities. The destination does not need the school-choice Python source to interpret the transition tree. It does need a compatible generic EDSL interpreter. The result-producing expression runs through the standard Machine close operation rather than an application-specific command dispatcher.

3 The state carried through iteration

The live allocation is one typed record $x = (Q, A, H, n_S, n_C, L, f)$:

Value	Interpretation
Q	student-proposing queue; unused by the school-proposing loop
$A : S \to C \cup \{\bot\}$	current student assignments
$H : C \to 2^S$	each school's tentative holds
$n_S : S \to \mathbb{N}$	each student's next school index
$n_C : C \to \mathbb{N}$	each school's next student-priority index
L	ordered proposal log
f	completion flag

Initial state is all students unmatched, all school holds empty, both cursor maps zeroed, an empty proposal log, and f = false. In each iteration the accumulator is a complete allocation record. The EDSL `iterate` operation binds the current record to a local name, evaluates the stop predicate, and if false evaluates one pure step expression to produce the next record. The step has a declared record type, checked after every internal iteration. If its explicit step budget is exhausted before the condition becomes true, the Runtime raises an execution error; it never silently returns a partial allocation.

4 Student-proposing loop

At each iteration, Q begins with the student IDs in fixed order. The step takes its first student s. If s has exhausted their list, the step removes s from Q. Otherwise it selects $c = \succ_s [n_S(s)]$, forms $H(c) \cup \{s\}$, sorts those candidates by c's priority rank, and retains the first q_c. It updates assignments and holds, advances $n_S(s)$, records the proposal, then puts rejected candidates at the end of Q. The loop stops when Q is empty.

Listing 2: Student-proposing loop as EDSL expressions, including queue choice, proposal effects, and stop condition

```

def _student_proposing_loop():
    """Return the serialized bounded DA iteration for student proposers."""
    state = local("matching")
    student = local("student")
    school = local("school")
    ranking = constant("student_preferences").get(student)
    cursor = state.get("student_next_choice").get(student)

    candidates = state.get("school_holds").get(school).appended(student)
    ranked_candidates = reduce(
        "sort_records",
        map_sequence(
            candidates,
            item="candidate",
            value_expr=record(
                student=local("candidate"),
                priority=constant("school_priority_ranks")
                .get(school)
                .get(local("candidate")),
            ),
        ),
        fields=("priority",),
        descending=(False,),
    )
```

```
    sorted_students = map_sequence(
        ranked_candidates,
        item="candidate",
        value_expr=local("candidate").get("student"),
    )
    retained = take(sorted_students, constant("capacities").get(school))
    rejected = map_sequence(
        filter_items(
            sorted_students,
            item="candidate",
            predicate=~retained.contains(local("candidate")),
        ),
        item="candidate",
        value_expr=local("candidate"),
    )

    proposal_state = let(
        "retained",
        retained,
        let(
            "rejected",
            rejected,
            record(
                queue=state.get("queue").drop_first() + local("rejected"),
                assignments=map_items(
                    state.get("assignments"),
                    key="candidate",
                    value="old_assignment",
                    key_expr=local("candidate"),
                    value_expr=choose(
                        candidates.contains(local("candidate")),
                        choose(
                            local("retained").contains(local("candidate")),
                            school,
                            None,
                        ),
                        local("old_assignment"),
                    ),
                ),
                school_holds=state.get("school_holds").with_item(
                    school, local("retained")
                ),
                student_next_choice=state.get("student_next_choice").with_item(
                    student, cursor + 1
                ),
                school_next_student=state.get("school_next_student"),
                proposal_log=state.get("proposal_log").appended(
                    record(
                        side=constant("proposer_side"),
                        proposer=student,
                        school=school,
                        outcome=choose(
                            local("retained").contains(student), "held", "rejected"
                        ),
                    )
                ),
                finished=False,
            ),
        ),
    )
    proposal = let(
        "school",
```

```
        ranking.at(cursor),
        let(
            "ranked_candidates",
            ranked_candidates,
            let("sorted_students", sorted_students, proposal_state),
        ),
    )
    step = let(
        "student",
        state.get("queue").first(),
        let(
            "ranking",
            constant("student_preferences").get(local("student")),
            let(
                "cursor",
                state.get("student_next_choice").get(local("student")),
                choose(
                    local("cursor") >= local("ranking").length(),
                    state.with_item("queue", state.get("queue").drop_first()),
                    proposal,
                ),
            ),
        ),
    )
    max_steps = (
        reduce(
            "sum",
            map_sequence(
                constant("students"),
                item="student_id",
                value_expr=constant("student_preferences")
                .get(local("student_id"))
                .length(),
            ),
        )
        + constant("students").length()
    )
    allocation = iterate(
        constant("initial_allocation"),
        state="matching",
        state_type=_allocation_type(),
        until=state.get("queue").length() == 0,
        step=step,
        max_steps=max_steps,
    )
```

Each proposal consumes one student ranking position. A student can have at most one final queue visit after exhausting their list. Therefore the machine derives a safe bound

$$B_S = \sum_{s \in S} | \succ_s | + |S|.$$

The bound is data-dependent and encoded as an expression over the serialized constants. It covers all proposals plus any final exhaustion visits.

5 School-proposing loop

At each iteration, the Machine filters schools in their declared order, keeping schools with an open seat and an untried student in their priority list. It selects the first active school c, then $s = \succ_c [n_C(c)]$. Student

s accepts if unmatched or if they prefer c to their current assignment. An accepted offer updates A and both affected schools' hold sets; a rejection leaves assignments and holds unchanged. The school's cursor advances either way, and the event is appended to L. Iteration stops when the active-school sequence is empty.

Listing 3: School-proposing loop as EDSL expressions, including eligibility selection and student choice

```

def _school_proposing_loop():
    """Return the serialized bounded DA iteration for school proposers."""
    state = local("matching")
    school = local("school")
    student = local("student")
    active_schools = filter_items(
        constant("schools"),
        item="candidate_school",
        predicate=(
            state.get("school_holds")
            .get(local("candidate_school"))
            .length()
            < constant("capacities").get(local("candidate_school"))
        )
        & (
            state.get("school_next_student").get(local("candidate_school"))
            < constant("school_priorities").get(local("candidate_school")).length()
        ),
    )
    priority_list = constant("school_priorities").get(school)
    cursor = state.get("school_next_student").get(school)
    current_school = state.get("assignments").get(student)
    accepted = choose(
        current_school == None,
        True,
        constant("student_ranks").get(student).get(school)
        < constant("student_ranks").get(student).get(current_school),
    )

    next_state = record(
        queue=[],
        assignments=state.get("assignments").with_item(
            student, choose(accepted, school, current_school)
        ),
        school_holds=map_items(
            state.get("school_holds"),
            key="school_key",
            value="held_students",
            key_expr=local("school_key"),
            value_expr=choose(
                local("school_key") == school,
                choose(
                    accepted,
                    local("held_students").appended(student),
                    local("held_students"),
                ),
                choose(
                    accepted & (local("school_key") == current_school),
                    filter_items(
                        local("held_students"),
                        item="held_student",
                        predicate=local("held_student") != student,
                    ),
                    local("held_students"),
```

```
                ),
            ),
        ),
        student_next_choice=state.get("student_next_choice"),
        school_next_student=state.get("school_next_student").with_item(
            school, cursor + 1
        ),
        proposal_log=state.get("proposal_log").appended(
            record(
                side=constant("proposer_side"),
                proposer=school,
                student=student,
                outcome=choose(accepted, "held", "rejected"),
            )
        ),
        finished=False,
    )
    step = let(
        "school",
        active_schools.first(),
        let(
            "student",
            priority_list.at(cursor),
            next_state,
        ),
    )
    max_steps = constant("schools").length() * constant("students").length()
    allocation = iterate(
        constant("initial_allocation"),
        state="matching",
        state_type=_allocation_type(),
        until=active_schools.length() == 0,
        step=step,
        max_steps=max_steps,
    )
```

Every school can propose to each student no more than once, so the loop has the finite upper bound

$$B_C = |C|\,|S|.$$

The stop predicate checks the current allocation for any school that has both remaining capacity and an untried priority candidate. The loop does not rely on a Python scheduler to make this eligibility decision.

6 Generic execution boundary

The run driver now performs only generic lifecycle operations:

Listing 4: Generic initialization and close; there is no proposal-selection loop

```
def run_machine(proposer_side: str, seed: int = 42017, student_preferences=None):
    machine = build_machine(
        seed=seed,
        proposer_side=proposer_side,
        student_preferences_override=student_preferences,
    )
    runtime = Runtime()
    state = runtime.initial_state(machine)
    result = runtime.close_result(machine, state)
    if result.event["status"] != "applied":
        raise RuntimeError(f"Machine close did not apply: {result.event}")
```

```
    state = result.state
    return machine, state
```

The close effect is atomic at the shared-state transition boundary: the Runtime evaluates the bounded loop against an internal accumulator and commits the resulting allocation as the close transition. The proposal log remains available in the final allocation for inspection. Intermediate matching states are not separately committed or externally observable during this close operation. That is a deliberate trade-off for this pilot: the whole matching is a pure, portable expression, while crash-resumable checkpoints between proposals would require a future resumable-iteration protocol.

The LLM interview is upstream of the Machine. It produces rankings that are validated and stored as constants. Provider calls, ranking normalization, and output analysis are not hidden inside the matching loop. For the LLM pilot the same saved profile and priority seed are embedded in both proposer-side definitions.

7 Observed experiment results

The preference pilot used twelve authored student personas, four school descriptions, and one `gpt-4o-mini` ranking interview per persona. School priorities use seed 42017. Both Machine instances executed through the new close-time loop.

Outcome	Student-proposing	School-proposing
First choices assigned	12 / 12	7 / 12
Mean student rank, 1 = best	1.00	1.75
Proposal iterations	12	22
Students assigned	12 / 12	12 / 12
Blocking pairs	0	0

The LLM-generated top choices happen to divide evenly: each school is the first choice of three personas, exactly its capacity. Student-proposing DA therefore settles after one proposal per student. School-proposing DA follows priority order and yields a different stable matching. This is a property of this one preference profile, not a general claim about school-choice markets or LLM preferences. Full rankings, proposals, and allocations are in `pilot_llm_01/results.json`.

8 What this achieves and what remains

The core goal is now met for this mechanism: the serialized Machine itself specifies who acts next, which proposal is made, how the state changes, how the loop terminates, and the maximum iteration budget. The Python runner no longer contains school-choice-specific control flow. A generic remote EDSL runner can initialize the Machine and perform the standard close operation, subject to implementing the same EDSL version and capabilities.

This is not yet a general workflow graph language. The loop is an expression in Machine close effects and computes to completion atomically. A future protocol could make long loops resumable between iterations while preserving the same declarative step/stop/bound structure. The current implementation is sufficient to remove the bespoke proposal scheduler and submit this school-choice experiment as data to a compatible generic executor.

The pilot's conformance file compares outcomes with independent textbook Python references, verifies stability and capacity, checks proposer-side comparisons, and checks JSON round-tripping. These tests were part of the existing study suite; this note describes the implementation and observed pilot outputs rather than claiming broader empirical validation. The synthetic personas and one response per persona cannot establish that the rankings represent real student preferences.

A Artifacts

- `machine_definition.py`: serializable Machine constants, allocation record type, and both embedded loops.
- `run_pilot.py`: generic Runtime initialization and Machine close call; independent outcome checks are post-run analysis.
- `llm_pilot.py`: persona interviews and preference validation.
- `pilot_llm_01/preferences.json`: exact LLM-generated rankings.
- `pilot_llm_01/results.json`: constants, proposal logs, assignments, and metrics.